LAB PROGRAM CHAPTER 3

1. Write a Program to find both the maximum and minimum values in the array. Implement using any

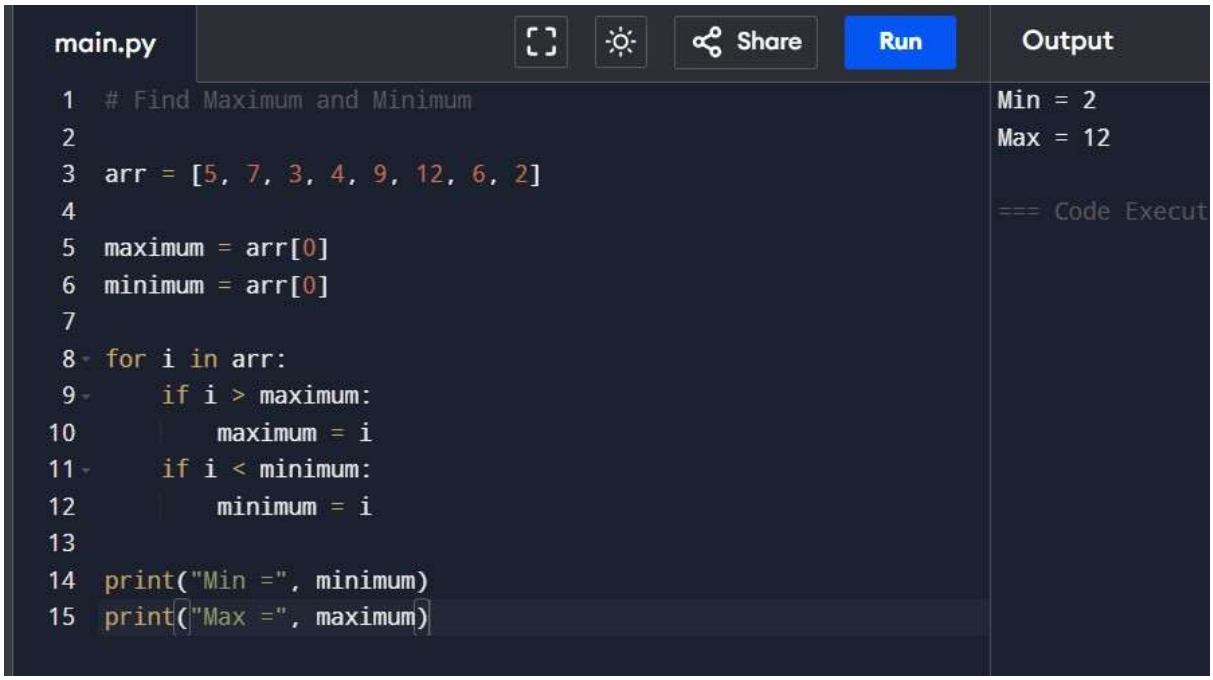
programming language of your choice. Execute your code and provide the maximum and

minimum values found.

CODE

```
arr = [5, 7, 3, 4, 9, 12, 6, 2]
maximum = arr[0]
minimum = arr[0]
for i in arr:
    if i > maximum:
        maximum = i
    if i < minimum:
        minimum = i
print("Min =", minimum)
print("Max =", maximum)
```

OUTPUT



2. Consider an array of integers sorted in ascending order: 2,4,6,8,10,12,14,18. Write a Program to

find both the maximum and minimum values in the array. Implement using any programming

language of your choice. Execute your code and provide the maximum and minimum values found.

CODE

arr = [2, 4, 6, 8, 10, 12, 14, 18]

minimum = arr[0] maximum

= arr[-1] print("Min =",

minimum) print("Max =",

maximum) **OUTPUT**

3. You are given an unsorted array 31,23,35,27,11,21,15,28. Write a program for Merge Sort and

implement using any programming language of your choice.

CODE

def merge_sort(arr): if

len(arr) > 1:

mid = len(arr) // 2

left = arr[:mid] right

= arr[mid:]

```
        merge_sort(left)
        merge_sort(right) i
        = j = k = 0
        while i < len(left) and j < len(right): if
            left[i] < right[j]:
                arr[k] = left[i] i
                += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left): arr[k]
            = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
arr = [31,23,35,27,11,21,15,28]
merge_sort(arr)
print("Sorted Array:")
print(arr)
```

OUTPUT

main.py

```
def merge_sort(arr):
    if len(arr) > 1:

        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

```

Output

```
Sorted Array:
[11, 15, 21, 23, 27, 28, 31, 35]

=== Code Execution Successful ===
```

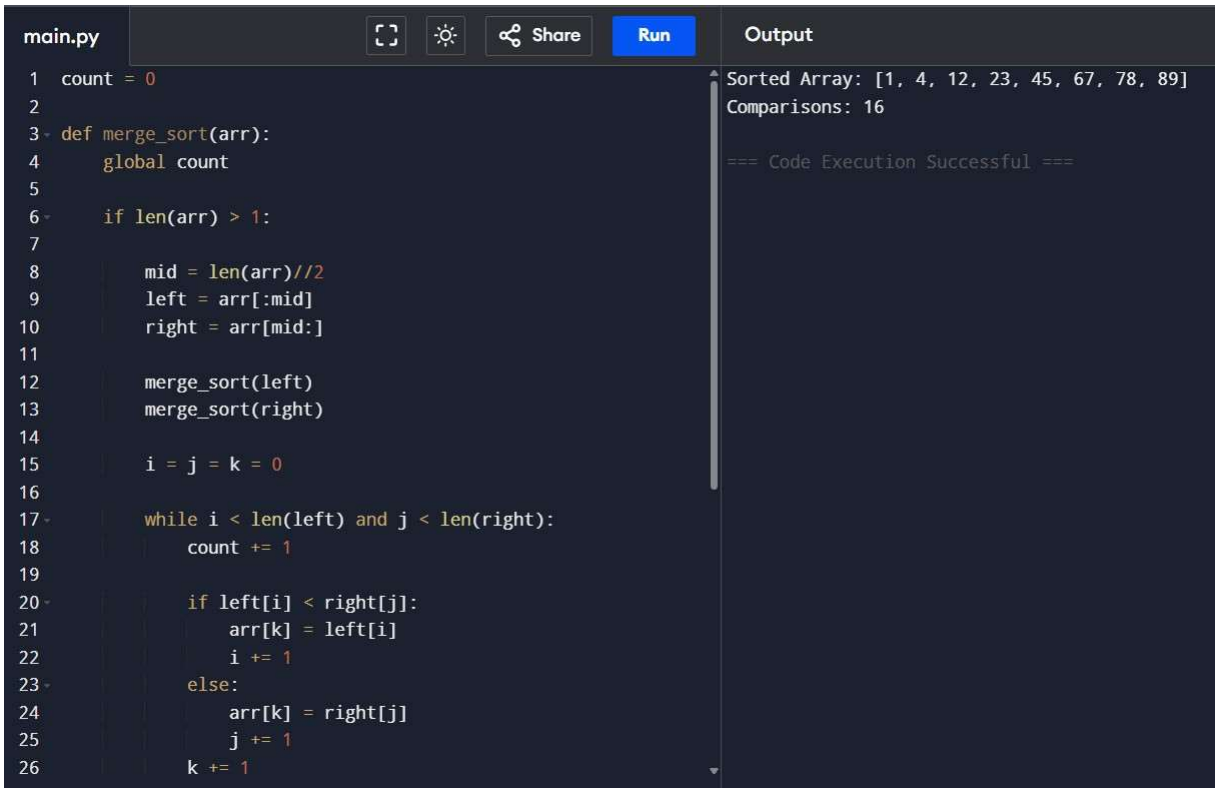
4. Implement the Merge Sort algorithm in a programming language of your choice and test it on the

array 12,4,78,23,45,67,89,1. Modify your implementation to count the number of comparisons made during the sorting process. Print this count along with the sorted array. CODE

```
count = 0
def merge_sort(arr):
    global count
    if len(arr) > 1:
        mid = len(arr)//2 left
        = arr[:mid] right =
        arr[mid:]
        merge_sort(left)
        merge_sort(right)
```

```
        i = j = k = 0
        while i < len(left) and j < len(right): count
            += 1
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left): arr[k]
            = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
arr = [12,4,78,23,45,67,89,1]
merge_sort(arr)  print("Sorted
Array:", arr)
print("Comparisons:", count)
```

OUTPUT



5. Given an unsorted array 10,16,8,12,15,6,3,9,5 Write a program to perform Quick Sort. Choose the

first element as the pivot and partition the array accordingly. Show the array after this partition.

Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call

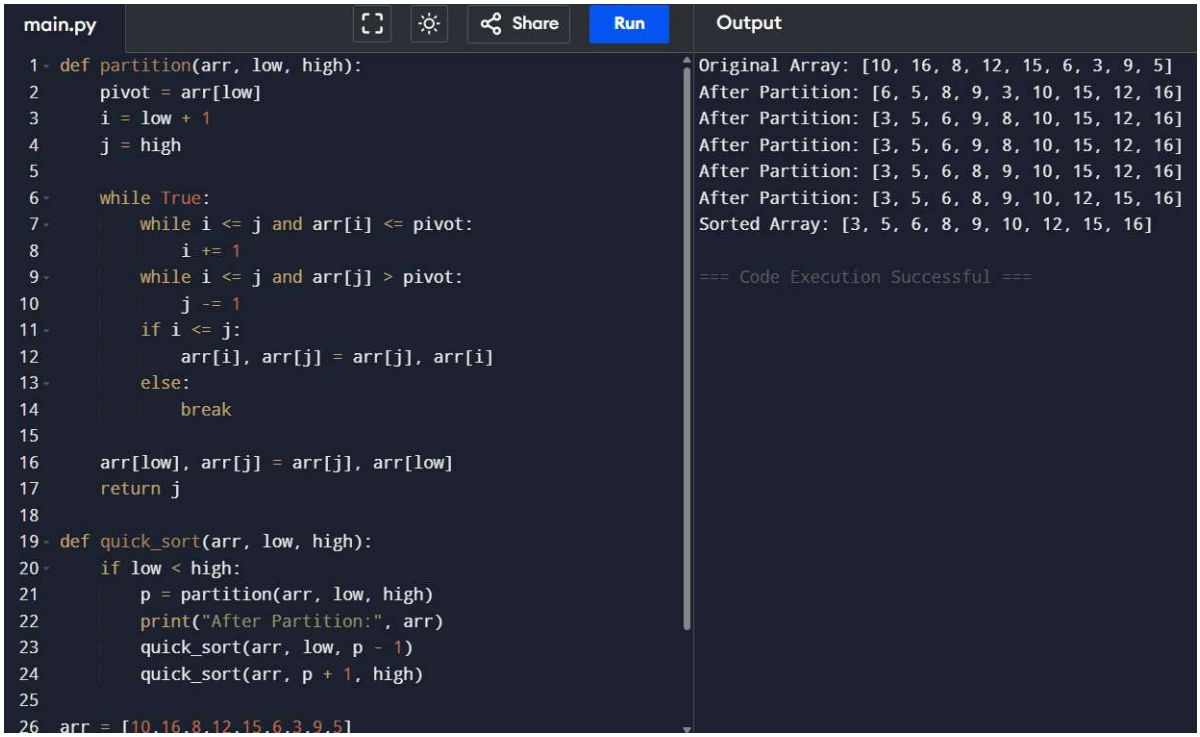
until the entire array is sorted.

CODE

```
def partition(arr, low, high):
    pivot = arr[low]
    i = low + 1
    j = high
    while True:
        while i <= j and arr[i] <= pivot: i
            += 1
        while i <= j and arr[j] > pivot: j
            -= 1
```

```
        if i <= j:
            arr[i], arr[j] = arr[j], arr[i]
        else:
            break
    arr[low], arr[j] = arr[j], arr[low]
    return j
def quick_sort(arr, low, high): if
    low < high:
        p = partition(arr, low, high) print("After
        Partition:", arr) quick_sort(arr, low, p -
        1) quick_sort(arr, p + 1, high)
arr = [10,16,8,12,15,6,3,9,5]
print("Original Array:", arr)
quick_sort(arr, 0, len(arr)-1)
print("Sorted Array:", arr)
```

OUTPUT



6. Implement the Quick Sort algorithm in a programming language of your choice and test it on the

array 19,72,35,46,58,91,22,31. Choose the middle element as the pivot and partition the array

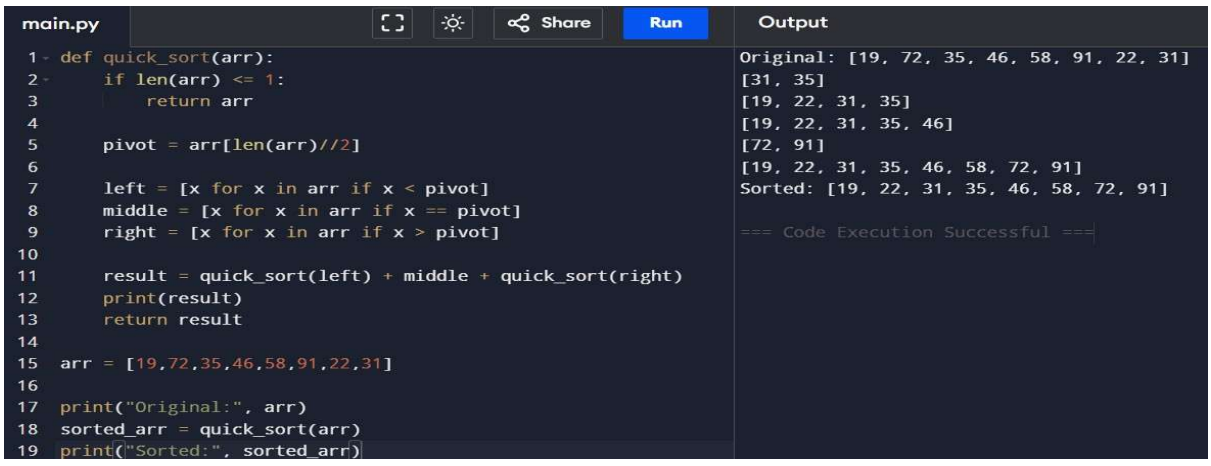
accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays

formed. Display the array after each recursive call until the entire array is sorted.
Execute your code

and show the sorted array. CODE

```
def quick_sort(arr): if
 len(arr) <= 1: return
            arr
    pivot = arr[len(arr)//2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    result = quick_sort(left) + middle + quick_sort(right) print(result)
    return result
arr = [19,72,35,46,58,91,22,31]
print("Original:", arr)
sorted_arr = quick_sort(arr)
print("Sorted:", sorted_arr)
```

OUTPUT



7. Implement the Binary Search algorithm in a programming language of your choice and test it on the

array 5,10,15,20,25,30,35,40,45 to find the position of the element 20. Execute your code and provide the index of the element 20. Modify your implementation to count the number of comparisons made during the search process. Print this count along with the result. CODE

```
def binary_search(arr, key):
    low = 0
    high = len(arr)-1
    comparisons = 0
    while low <= high:
        comparisons += 1 mid
        = (low + high)//2 if
        arr[mid] == key:
            return mid + 1, comparisons elif
        arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1, comparisons
arr = [5,10,15,20,25,30,35,40,45]
key = 20
pos, count = binary_search(arr,key)
print("Position:", pos) print("Comparisons:",
count)
```

OUTPUT

main.py

```
def binary_search(arr, key):
    low = 0
    high = len(arr)-1
    comparisons = 0

    while low <= high:
        comparisons += 1
        mid = (low + high)//2

        if arr[mid] == key:
            return mid + 1, comparisons
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1, comparisons

arr = [5,10,15,20,25,30,35,40,45]
key = 20

pos, count = binary_search(arr,key)

print("Position:", pos)
print("Comparisons:", count)
```

Output

```
Position: 4
Comparisons: 4

=== Code Execution Succes
```

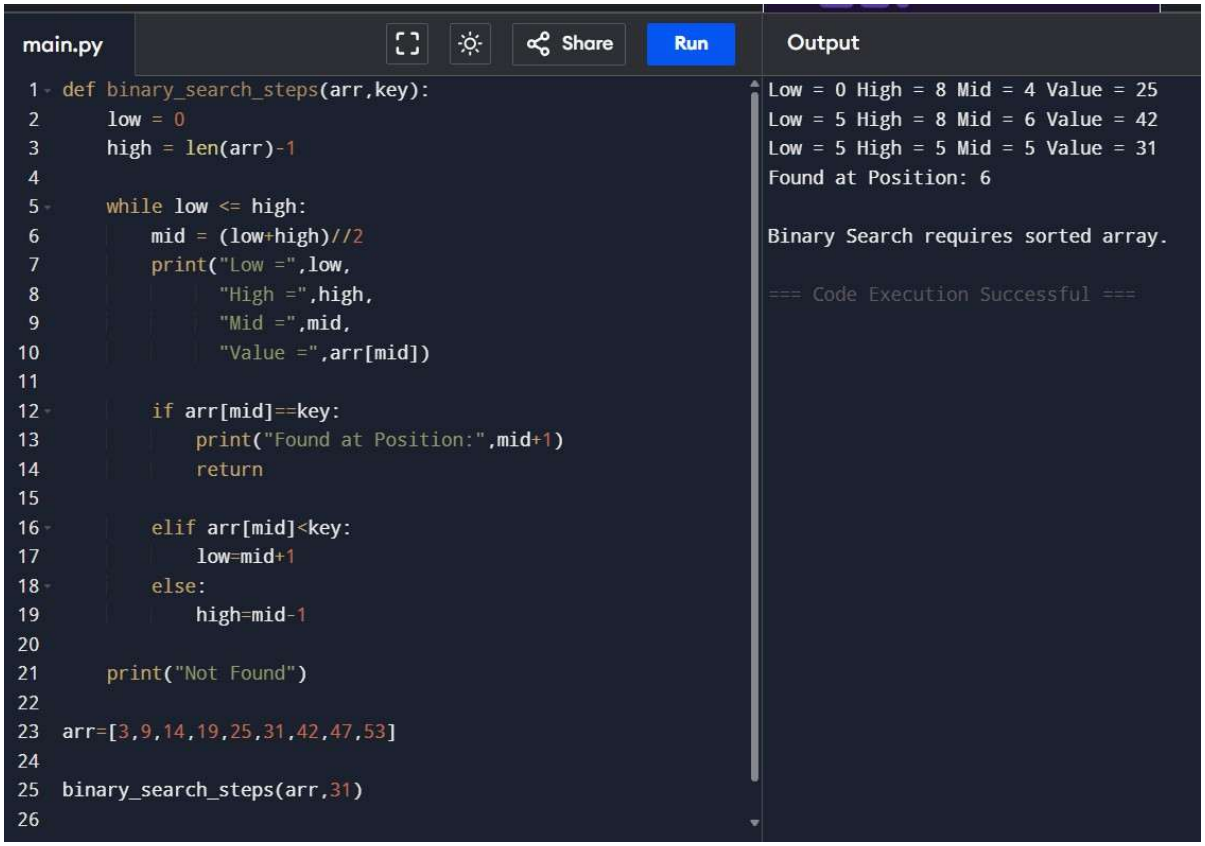
8. You are given a sorted array 3,9,14,19,25,31,42,47,53 and asked to find the position of the element

31 using Binary Search. Show the mid-point calculations and the steps involved in finding the element. Display, what would happen if the array was not sorted, how would this impact the performance and correctness of the Binary Search algorithm? CODE

```
def binary_search_steps(arr,key): low
    = 0
    high = len(arr)-1
    while low <= high:
        mid = (low+high)//2
        print("Low =",low,
              "High =",high,
              "Mid =",mid,
```

```
            "Value =",arr[mid])
        if arr[mid]==key:
            print("Found at Position:",mid+1)
            return
        elif arr[mid]<key:
            low=mid+1
        else:
            high=mid-1
    print("Not Found")
arr=[3,9,14,19,25,31,42,47,53
binary_search_steps(arr,31)
print("\nBinary Search requires sorted array.")
```

OUTPUT



9. Given an array of points where points[i] = [xi, yi] represents a point on the X-Y plane and

an integer k, return the k closest points to the origin (0, 0). CODE

```
import heapq

def kClosest(points,k): return

    heapq.nsmallest(

        k,

        points,

        key=lambda p:p[0]**2+p[1]**2

    )

points=[[1,3],[-2,2],[5,8],[0,1]]

k=2

print(kClosest(points,k))
```

OUTPUT

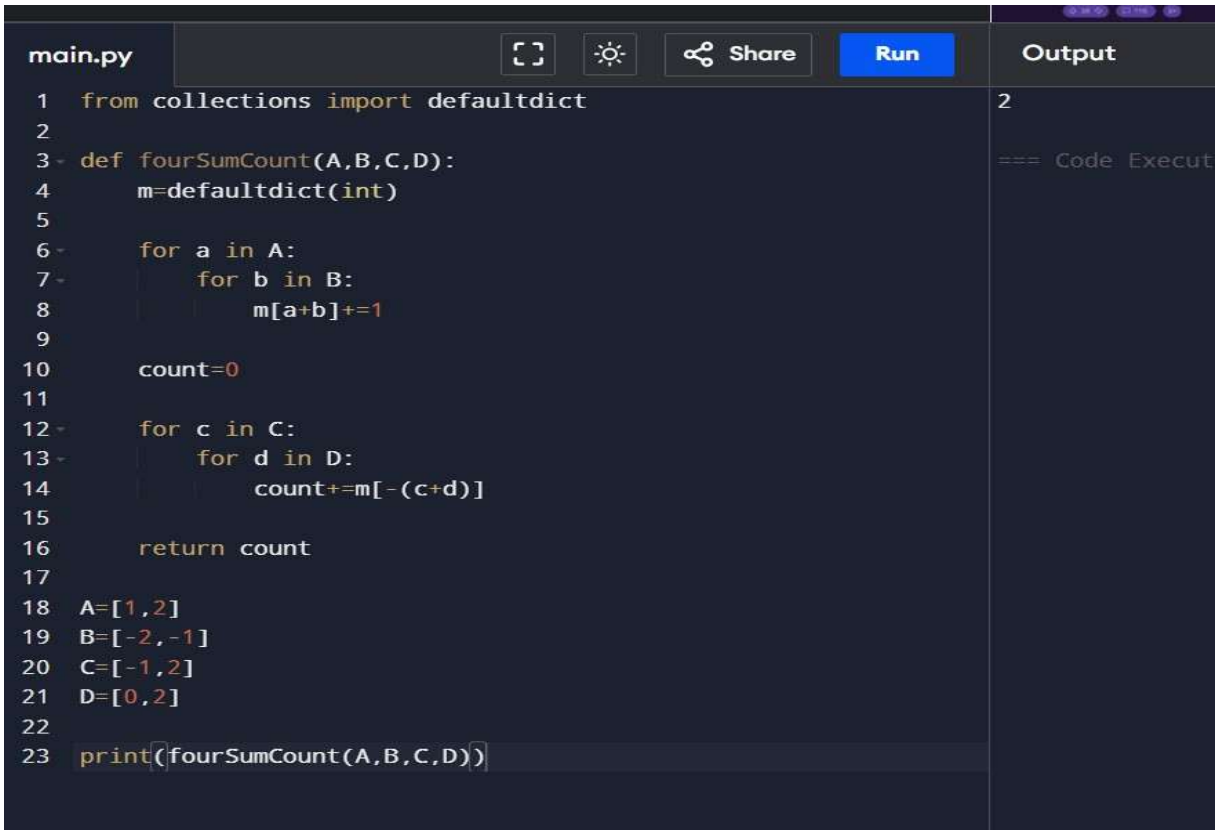
10. Given four lists A, B, C, D of integer values,Write a program to compute how many tuples n(i, j, k, l) there are such that A[i] + B[j] + C[k] + D[l] is zero.

CODE

```
from collections import defaultdict


def fourSumCount(A,B,C,D):
```

```
    m=defaultdict(int)
    for a in A:
        for b in B:
            m[a+b]+=1
    count=0
    for c in C:
        for d in D:
            count+=m[-(c+d)]
    return count
A=[1,2]
B=[-2,-1]
C=[-1,2]
D=[0,2]
print(fourSumCount(A,B,C,D))
```

OUTPUT



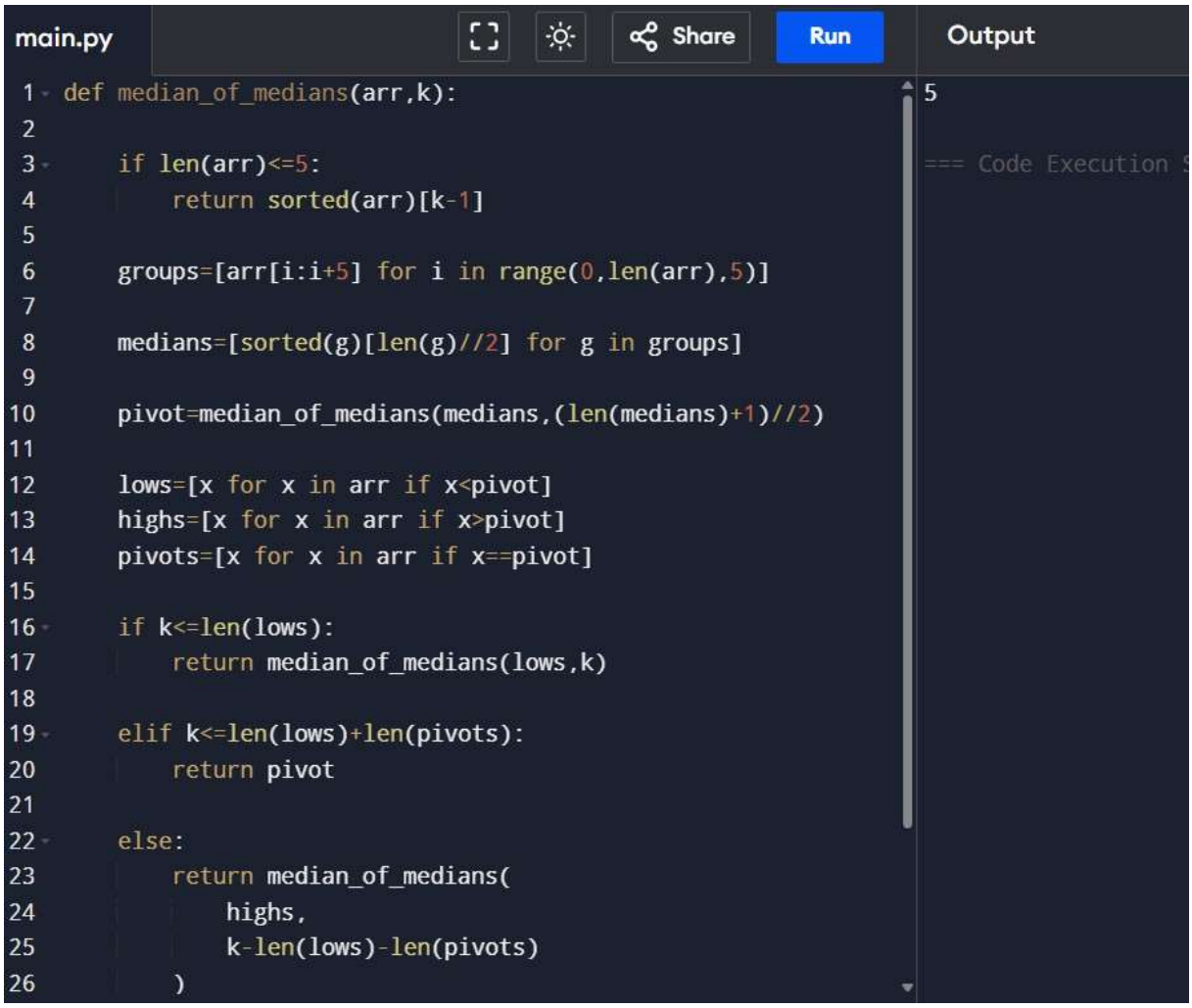
11. To Implement the Median of Medians algorithm ensures that you handle the worst-case

time complexity efficiently while finding the k-th smallest element in an unsorted array.

CODE

```
def median_of_medians(arr,k): if
    len(arr)<=5:
        return sorted(arr)[k-1]
    groups=[arr[i:i+5] for i in range(0,len(arr),5)]
    medians=[sorted(g)[len(g)//2] for g in groups]
    pivot=median_of_medians(medians,(len(medians)+1)//2) lows=[x
    for x in arr if x<pivot]
    highs=[x for x in arr if x>pivot]
    pivots=[x for x in arr if x==pivot]
    if k<=len(lows):
        return median_of_medians(lows,k) elif
    k<=len(lows)+len(pivots):
        return pivot
    else:
        return median_of_medians(
            highs,
            k-len(lows)-len(pivots)
        )

arr=[12,3,5,7,19]
k=2 print(median_of_medians(arr,k))
```

OUTPUT

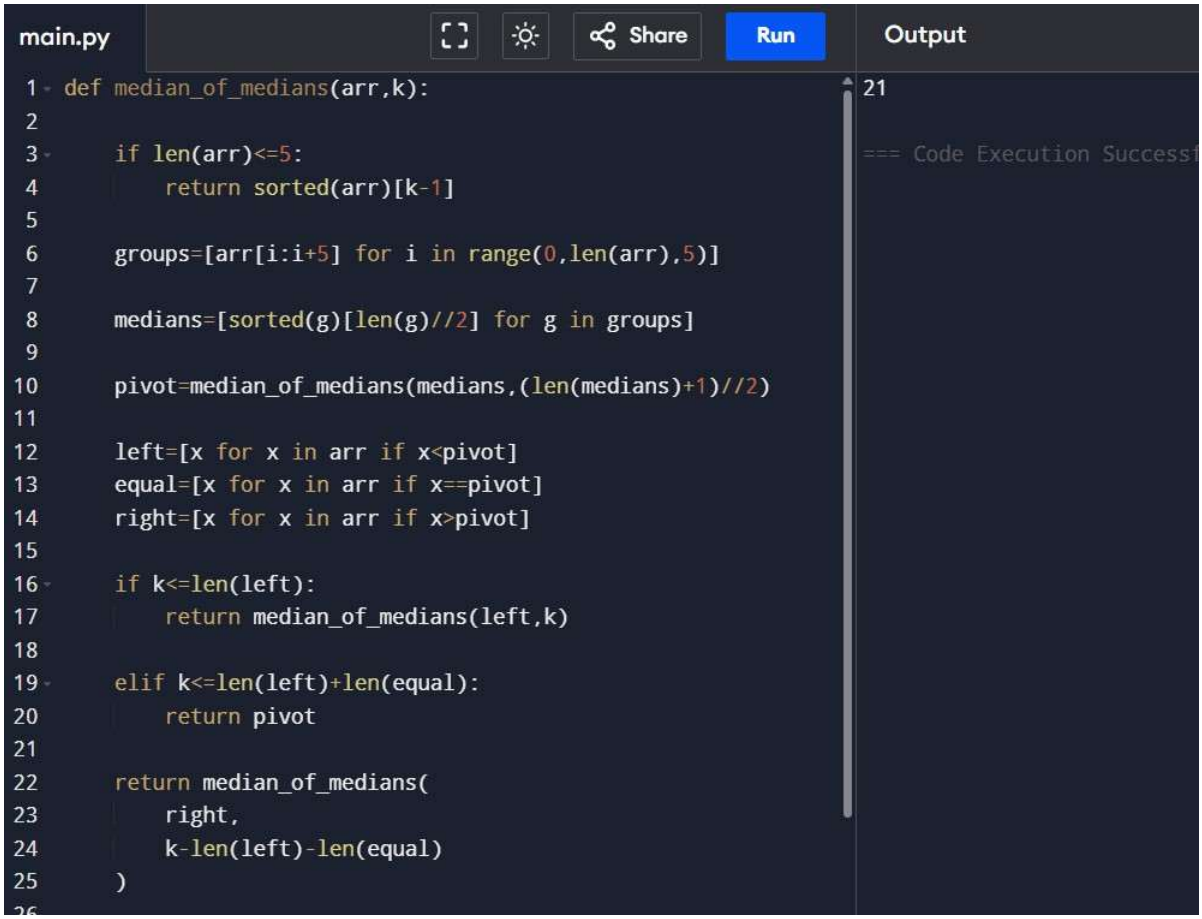


12. To Implement a function median_of_medians(arr, k) that takes an unsorted CODE

```
def median_of_medians(arr,k): if
    len(arr)<=5:
        return sorted(arr)[k-1]
    groups=[arr[i:i+5] for i in range(0,len(arr),5)]
    medians=[sorted(g)[len(g)//2] for g in groups]
    pivot=median_of_medians(medians,(len(medians)+1)//2) left=[x for
    x in arr if x<pivot]
    equal=[x for x in arr if x==pivot]
    right=[x for x in arr if x>pivot]

    if k<=len(left):
        return median_of_medians(left,k)
```

```
    elif k<=len(left)+len(equal): return
        pivot

    return median_of_medians( right,
        k-len(left)-len(equal)
    ) arr=[23,17,31,44,55,21,20,18,19,27]

print(median_of_medians(arr,5))
```

OUTPUT



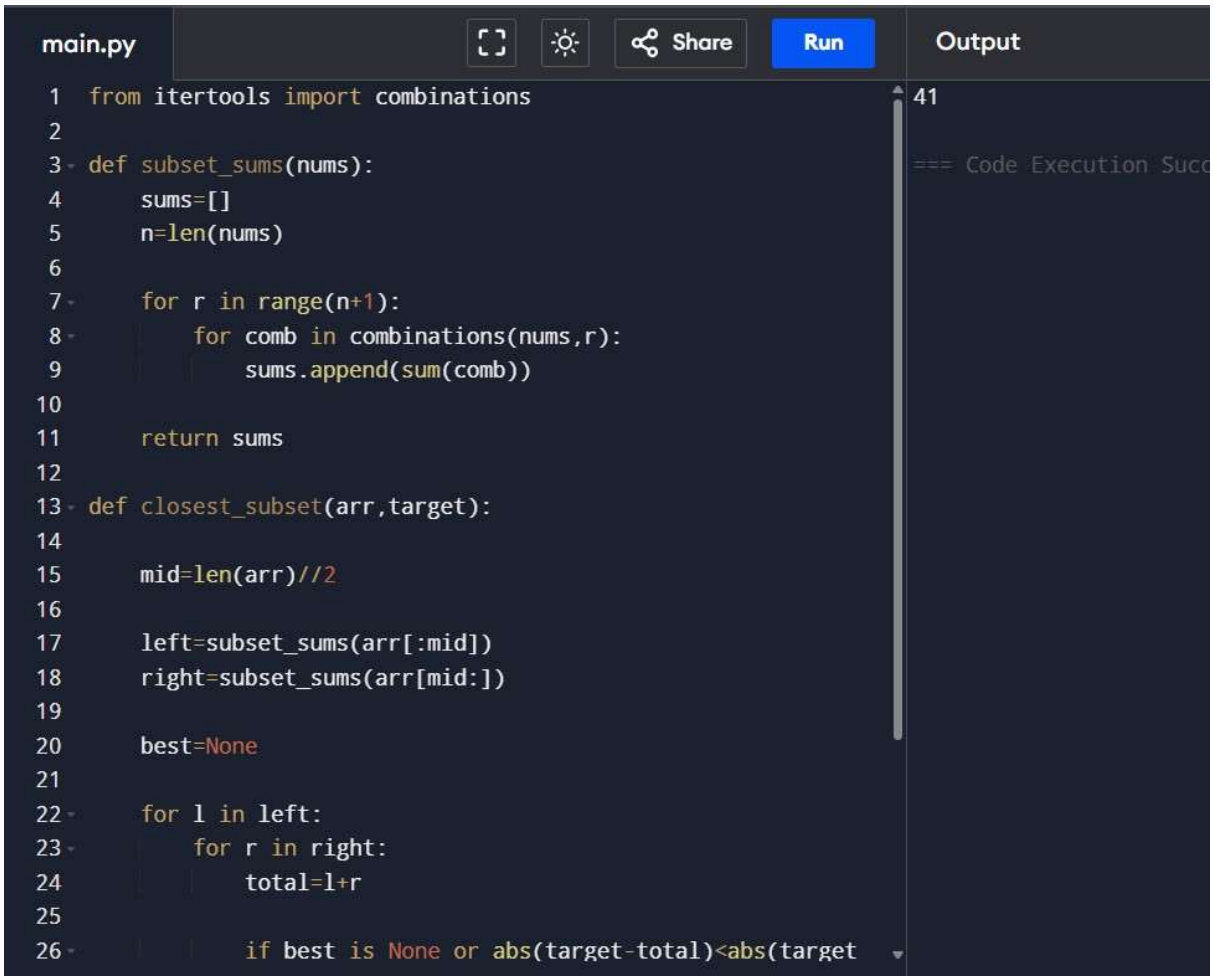
13. Write a program to implement Meet in the Middle Technique. Given an array of integers

and a target sum, find the subset whose sum is closest to the target. You will use the Meet

in the Middle technique to efficiently find this subset. CODE

```
from itertools import combinations
def subset_sums(nums):
    sums=[]
    n=len(nums)
    for r in range(n+1):
        for comb in combinations(nums,r): sums.append(sum(comb))
    return sums
def closest_subset(arr,target): mid=len(arr)//2

    left=subset_sums(arr[:mid])
    right=subset_sums(arr[mid:])

    best=None
    for l in left:
        for r in right:
            total=l+r

            if best is None or abs(target-total)<abs(target-best): best=total
    return best
arr=[45,34,4,12,5,2]

print(closest_subset(arr,42))
```

OUTPUT

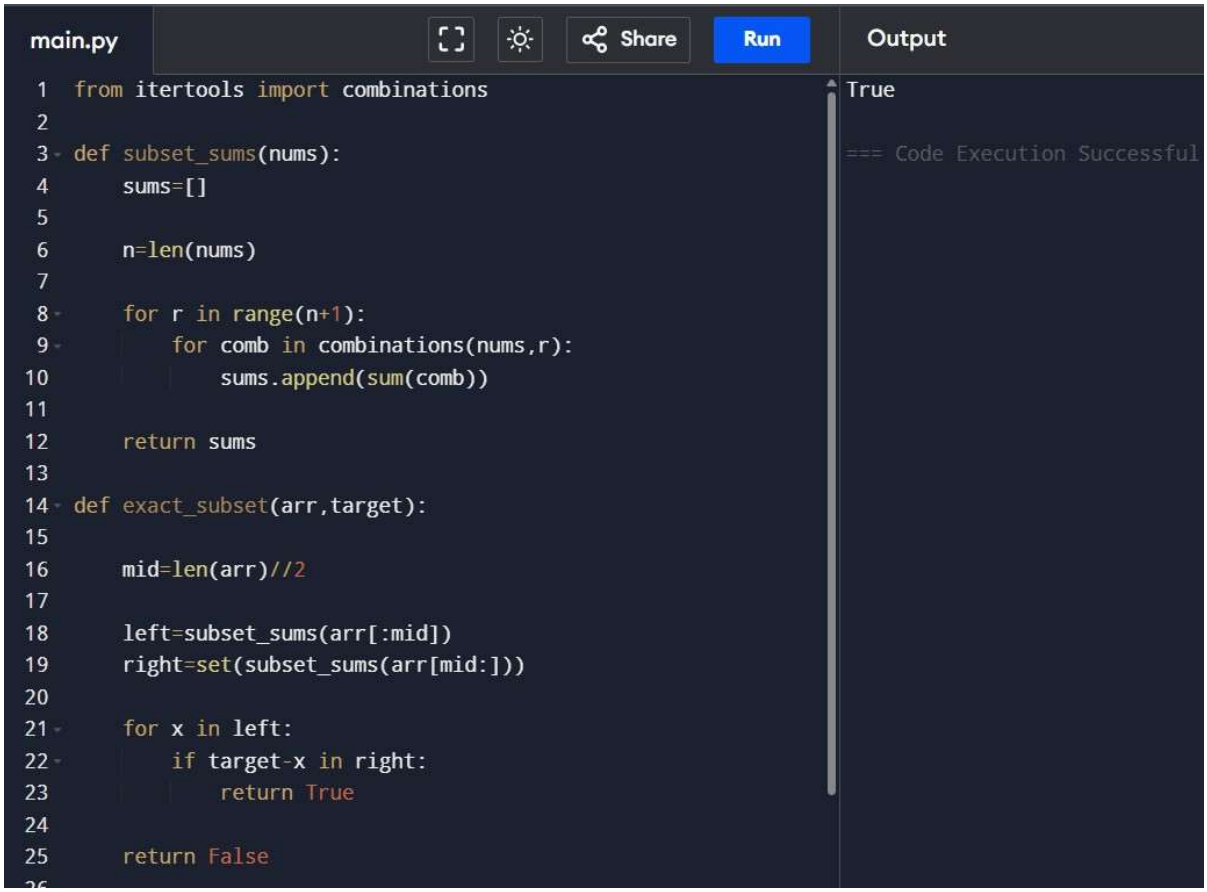


14. Write a program to implement Meet in the Middle Technique. Given a large array of integers and an exact sum E, determine if there is any subset that sums exactly to E. Utilize the Meet in the Middle technique to handle the potentially large size of the array. Return true if there is a subset that sums exactly to E, otherwise return false. CODE

```
from itertools import combinations
def subset_sums(nums):
    sums=[]
    n=len(nums)
    for r in range(n+1):
        for comb in combinations(nums,r): sums.append(sum(comb))
    return sums
def exact_subset(arr,target):
```

```
    mid=len(arr)//2
    left=subset_sums(arr[:mid])
    right=set(subset_sums(arr[mid:]))

    for x in left:
    if target-x in right: return
                    True

     return False

 arr=[1,3,9,2,7,12]
  print(exact_subset(arr,15))
```

OUTPUT



15 Given two 2×2 Matrices A and B

A=(1 7 B=(1 3

3 5) 7 5)

Use Strassen's matrix multiplication algorithm to compute the product matrix C such that C=A×B.

CODE

```
def strassen(A,B):

    a,b=A[0]
    c,d=A[1]

    e,f=B[0]
    g,h=B[1]

    p1=a*(f-h)
    p2=(a+b)*h
    p3=(c+d)*e
    p4=d*(g-e)
    p5=(a+d)*(e+h)
    p6=(b-d)*(g+h)
    p7=(a-c)*(e+f)
    c11=p5+p4-p2+p6
    c12=p1+p2
    c21=p3+p4
    c22=p1+p5-p3-p7
    return [[c11,c12],[c21,c22]]
A=[[1,7],
   [3,5]]

B=[[1,3],
   [7,5]]
C=strassen(A,B) for
row in C:
```

print(row)

OUTPUT